# Threats to validity, and what each control leaves behind

Shaded by what remains after the control rather than by how the threat was described

| | THREAT | CONTROL APPLIED | WHAT REMAINS |
|---|---|---|---|
| **INTERNAL** Is the language really the only difference? | Implementation bias between the two versions | Frozen specification, shared allocator and optimisation level, pinned hash functions, forced-idiom log, transcript parity | Single-author skill asymmetry. Parity bounds behaviour, not performance |
| **CONSTRUCT** Do the metrics measure what the questions ask? | The sweep measures the design, not the language | The operation is measured apart from the handoff and both are reported | One operation on one data structure is narrower than the sweep it supplements |
| | A defect catalogue is a sample the author chose | Two classes drawn from outside the guarantee, with expectations recorded in advance | Seven programs remain seven programs |
| | RQ3 captures resource cost but not its origin | None available on this host | The memory result is established but unexplained |
| | RQ4 measures the seam, not the safety of the whole | The distinction is restated wherever the result is quoted | One address space: a compile-time property of the Rust side bounds nothing at run time |
| **CONCLUSION** Are the differences real, or noise? | Run-to-run non-determinism | 20 interleaved repeats, bootstrap intervals, an exact paired permutation test, Holm correction, floors quoted | Pairing cancels only what a block shared, so the tail percentiles stay unresolved |
| | A corrected p-value that does not replicate | The allocator control runs every comparison twice, and replication is followed over the p-value | Costs a result where the two disagree: the CPU effect is reported as unresolved |
| **EXTERNAL** How far do the results travel? | One subsystem, one workload, one machine | RQ2 and RQ4 rest on the compilers and the C ABI rather than on the host | The performance results do not travel: shared mutable state or allocation churn could behave differently |

**Two of these are load-bearing**

The study cannot rule out single-author skill asymmetry, and its end-to-end instrument answers the question a vendor sizing a node would ask rather than the one a language comparison asks.

Only the last column is a live risk. The middle column is what has already been done about each.
Two rows carry no mitigation at all, and are recorded as such.